KodetoCareer — Development Specification

Version 1.0 | For Engineering Team

---

1. Project Structure

1.1 Backend (Node.js / Express.js)

```
kodetocareer-api/
■■■ src/
■   ■■■ config/
■   ■   ■■■ database.ts        # Prisma client singleton
■   ■   ■■■ redis.ts           # Redis client
■   ■   ■■■ aws.ts             # AWS SDK configuration
■   ■   ■■■ email.ts           # Nodemailer / SES config
■   ■   ■■■ app.ts             # Express app setup
■   ■
■   ■■■ middleware/
■   ■   ■■■ auth.ts            # JWT verification middleware
■   ■   ■■■ rbac.ts            # Role-based access control
■   ■   ■■■ rateLimiter.ts     # Rate limiting per route
■   ■   ■■■ errorHandler.ts    # Global error handler
■   ■   ■■■ auditLog.ts        # Auto-audit-log write operations
■   ■   ■■■ validate.ts        # Zod request validation
■   ■
■   ■■■ modules/
■   ■   ■■■ auth/
■   ■   ■   ■■■ auth.router.ts
■   ■   ■   ■■■ auth.controller.ts
■   ■   ■   ■■■ auth.service.ts
■   ■   ■   ■■■ auth.schema.ts     # Zod schemas
■   ■   ■
■   ■   ■■■ users/
■   ■   ■   ■■■ users.router.ts
■   ■   ■   ■■■ users.controller.ts
■   ■   ■   ■■■ users.service.ts
■   ■   ■
■   ■   ■■■ colleges/
■   ■   ■   ■■■ (same pattern)
■   ■   ■
■   ■   ■■■ students/
■   ■   ■■■ trainers/
■   ■   ■■■ courses/
■   ■   ■■■ modules-lessons/
■   ■   ■■■ batches/
```

```
■   ■   ■■■ attendance/
■   ■   ■■■ quizzes/
■   ■   ■■■ assignments/
■   ■   ■■■ certificates/
■   ■   ■■■ placements/
■   ■   ■■■ jobs/
■   ■   ■■■ notifications/
■   ■   ■■■ analytics/
■   ■
■   ■■■ jobs/                  # Bull queue workers
■   ■   ■■■ queues.ts          # Queue definitions
■   ■   ■■■ video.worker.ts
■   ■   ■■■ certificate.worker.ts
■   ■   ■■■ email.worker.ts
■   ■   ■■■ sms.worker.ts
■   ■   ■■■ report.worker.ts
■   ■
■   ■■■ services/              # Shared services
■   ■   ■■■ storage.service.ts     # S3 operations
■   ■   ■■■ email.service.ts       # Template rendering + send
■   ■   ■■■ sms.service.ts         # MSG91 integration
■   ■   ■■■ fcm.service.ts         # Firebase push notifications
■   ■   ■■■ pdf.service.ts         # Certificate PDF generation
■   ■   ■■■ analytics.service.ts   # Analytics aggregation
■   ■
■   ■■■ utils/
■   ■   ■■■ response.ts            # Standard API response helpers
■   ■   ■■■ pagination.ts          # Pagination utilities
■   ■   ■■■ codeGenerator.ts       # Student code, cert code generation
■   ■   ■■■ csvParser.ts           # CSV import parsing + validation
■   ■   ■■■ logger.ts              # Winston logger
■   ■
■   ■■■ index.ts                  # Entry point
■
■■■ prisma/
■   ■■■ schema.prisma
■   ■■■ migrations/
■   ■■■ seed.ts
■
■■■ tests/
■   ■■■ unit/
■   ■■■ integration/
■
■■■ .env.example
■■■ docker-compose.yml
■■■ Dockerfile
■■■ package.json
```

1.2 Frontend Web (React.js)

```
kodetocareer-web/
■■■ src/
```

```
■   ■■■ assets/
■   ■   ■■■ images/
■   ■   ■■■ icons/
■   ■
■   ■■■ components/
■   ■   ■■■ ui/                    # shadcn/ui base components
■   ■   ■■■ common/                # Shared application components
■   ■   ■   ■■■ DataTable.tsx
■   ■   ■   ■■■ PageHeader.tsx
■   ■   ■   ■■■ StatCard.tsx
■   ■   ■   ■■■ ProgressBar.tsx
■   ■   ■   ■■■ Badge.tsx
■   ■   ■   ■■■ EmptyState.tsx
■   ■   ■   ■■■ LoadingSkeleton.tsx
■   ■   ■   ■■■ ConfirmDialog.tsx
■   ■   ■■■ layout/
■   ■       ■■■ Sidebar.tsx
■   ■       ■■■ Header.tsx
■   ■       ■■■ PageContainer.tsx
■   ■
■   ■■■ pages/
■   ■   ■■■ auth/
■   ■   ■   ■■■ LoginPage.tsx
■   ■   ■   ■■■ ForgotPasswordPage.tsx
■   ■   ■■■ dashboard/
■   ■   ■   ■■■ SuperAdminDashboard.tsx
■   ■   ■■■ colleges/
■   ■   ■   ■■■ CollegeListPage.tsx
■   ■   ■   ■■■ CollegeDetailPage.tsx
■   ■   ■■■ students/
■   ■   ■   ■■■ StudentListPage.tsx
■   ■   ■   ■■■ StudentDetailPage.tsx
■   ■   ■■■ trainers/
■   ■   ■■■ courses/
■   ■   ■   ■■■ CourseListPage.tsx
■   ■   ■   ■■■ CourseDetailPage.tsx
■   ■   ■   ■■■ CourseBuilderPage.tsx
■   ■   ■■■ batches/
■   ■   ■■■ attendance/
■   ■   ■■■ assessments/
■   ■   ■■■ certificates/
■   ■   ■■■ placements/
■   ■   ■■■ jobs/
■   ■   ■■■ communication/
■   ■   ■■■ analytics/
■   ■   ■■■ college-admin/         # College admin portal pages
■   ■
■   ■■■ hooks/
■   ■   ■■■ useAuth.ts
■   ■   ■■■ useStudents.ts
■   ■   ■■■ useBatches.ts
■   ■   ■■■ ... (per domain)
■   ■
■   ■■■ services/
■   ■   ■■■ api.ts                 # Axios instance + interceptors
■   ■   ■■■ ... (per domain)
```

```
■   ■
■   ■■■ store/
■   ■   ■■■ authStore.ts           # Zustand auth state
■   ■
■   ■■■ types/
■   ■   ■■■ index.ts               # TypeScript interfaces
■   ■
■   ■■■ App.tsx
■
■■■ index.html
■■■ vite.config.ts
```

1.3 Mobile App (Flutter)

```
kodetocareer_app/
■■■ lib/
■   ■■■ main.dart
■   ■■■ app.dart
■   ■
■   ■■■ core/
■   ■   ■■■ api/
■   ■   ■   ■■■ api_client.dart       # Dio instance with interceptors
■   ■   ■   ■■■ api_endpoints.dart    # URL constants
■   ■   ■■■ storage/
■   ■   ■   ■■■ secure_storage.dart   # flutter_secure_storage wrapper
■   ■   ■■■ theme/
■   ■   ■   ■■■ app_theme.dart
■   ■   ■   ■■■ app_colors.dart
■   ■   ■   ■■■ app_text_styles.dart
■   ■   ■■■ router/
■   ■   ■   ■■■ app_router.dart       # go_router configuration
■   ■   ■■■ utils/
■   ■       ■■■ date_utils.dart
■   ■       ■■■ validators.dart
■   ■
■   ■■■ features/
■   ■   ■■■ auth/
■   ■   ■   ■■■ data/
■   ■   ■   ■   ■■■ auth_repository.dart
■   ■   ■   ■   ■■■ auth_api.dart
■   ■   ■   ■■■ domain/
■   ■   ■   ■   ■■■ auth_models.dart
■   ■   ■   ■■■ presentation/
■   ■   ■   ■   ■■■ screens/
■   ■   ■   ■   ■   ■■■ splash_screen.dart
■   ■   ■   ■   ■   ■■■ onboarding_screen.dart
■   ■   ■   ■   ■   ■■■ login_screen.dart
■   ■   ■   ■   ■   ■■■ register_screen.dart
■   ■   ■   ■   ■   ■■■ email_verify_screen.dart
■   ■   ■   ■   ■   ■■■ profile_setup_screen.dart
■   ■   ■   ■   ■■■ providers/
■   ■   ■   ■       ■■■ auth_provider.dart
```

```
■   ■   ■   ■■■ widgets/
■   ■   ■
■   ■   ■■■ dashboard/
■   ■   ■■■ courses/
■   ■   ■■■ lessons/
■   ■   ■■■ quizzes/
■   ■   ■■■ assignments/
■   ■   ■■■ attendance/
■   ■   ■■■ placement/
■   ■   ■■■ certificates/
■   ■   ■■■ profile/
■   ■   ■■■ notifications/
■   ■
■   ■■■ shared/
■       ■■■ widgets/
■       ■   ■■■ primary_button.dart
■       ■   ■■■ secondary_button.dart
■       ■   ■■■ custom_text_field.dart
■       ■   ■■■ loading_overlay.dart
■       ■   ■■■ error_widget.dart
■       ■   ■■■ skeleton_loader.dart
■       ■   ■■■ progress_bar.dart
■       ■■■ models/
■           ■■■ api_response.dart
■
■■■ assets/
■   ■■■ images/
■   ■■■ icons/
■   ■■■ lottie/              # Celebration animations
■   ■■■ fonts/
■       ■■■ Inter/
■
■■■ android/
■■■ ios/
■■■ pubspec.yaml
```

---

2. Code Standards

2.1 Backend Code Standards

TypeScript:

- `strict: true` in tsconfig
- No `any` types — use `unknown` where type is genuinely unknown
- All API inputs validated with Zod before controller logic

Service Layer Pattern:

```
// Controller: HTTP concerns only (parse request, call service, format response)
```

```
export const createBatch = async (req: Request, res: Response) => {
  const data = createBatchSchema.parse(req.body);
  const batch = await batchService.createBatch(data, req.user.id);
  return res.status(201).json(successResponse(batch));
};

// Service: Business logic only (no HTTP, no direct DB queries)
export const createBatch = async (data: CreateBatchDto, createdBy: string) => {
  const college = await prisma.college.findUnique({ where: { id: data.collegeId } });
  if (!college) throw new AppError('College not found', 404);

  const batch = await prisma.batch.create({ data: { ...data, createdBy } });
  await notificationService.notify({ type: 'BATCH_CREATED', batchId: batch.id });
  return batch;
};
```

Error Handling:

```
// Custom error class
class AppError extends Error {
  constructor(
    public message: string,
    public statusCode: number = 500,
    public code: string = 'INTERNAL_ERROR'
  ) {
    super(message);
  }
}

// Global error handler in middleware
export const errorHandler = (err: Error, req: Request, res: Response) => {
  if (err instanceof AppError) {
    return res.status(err.statusCode).json(errorResponse(err.message, err.code));
  }
  if (err instanceof ZodError) {
    return res.status(400).json(validationErrorResponse(err.errors));
  }
  logger.error('Unhandled error', { error: err, path: req.path });
  return res.status(500).json(errorResponse('Something went wrong'));
};
```

Prisma Query Guidelines:

```
// Always select only needed fields
const students = await prisma.student.findMany({
  where: { collegeId, deletedAt: null },
  select: {
    id: true,
    studentCode: true,
    user: { select: { firstName: true, lastName: true, email: true } },
    placementStatus: true,
  },
  orderBy: { createdAt: 'desc' },
  skip: offset,
  take: limit,
});
```

```
// Never do N+1 queries — use include or separate batch queries
```

2.2 Flutter Code Standards

Feature-First Architecture:

Each feature is self-contained with its own data, domain, and presentation layers. Shared widgets go in `/shared/widgets`.

Riverpod Providers:

```
// Repository provider
final authRepositoryProvider = Provider<AuthRepository>((ref) {
  return AuthRepository(ref.read(apiClientProvider));
});

// State notifier for complex state
final authProvider = StateNotifierProvider<AuthNotifier, AuthState>((ref) {
  return AuthNotifier(ref.read(authRepositoryProvider));
});

// AsyncNotifier for data fetching
final coursesProvider = AsyncNotifierProvider<CoursesNotifier, List<Course>>(() {
  return CoursesNotifier();
});
```

Error Handling Pattern:

```
// In screens, always handle loading, error, and data states
ref.watch(coursesProvider).when(
  loading: () => const SkeletonLoader(),
  error: (err, stack) => ErrorWidget(
    message: 'Failed to load courses',
    onRetry: () => ref.refresh(coursesProvider),
  ),
  data: (courses) => CourseList(courses: courses),
);
```

2.3 React Code Standards

Component Pattern:

```
// Feature component with React Query
const StudentList: React.FC = () => {
  const { data, isLoading, error } = useQuery({
    queryKey: ['students', filters],
    queryFn: () => studentsApi.getStudents(filters),
  });

  if (isLoading) return <StudentListSkeleton />;
  if (error) return <ErrorState message="Failed to load students" />;
  if (!data?.length) return <EmptyState title="No students found" />;

  return <DataTable data={data} columns={studentColumns} />;
```

```
};
```

No prop drilling beyond 2 levels. Use React Query cache or Zustand for shared state.

---

3. API Specification Details

3.1 Authentication Middleware

```
// Middleware applied to all protected routes
export const authenticate = async (req: Request, res: Response, next: NextFunction) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) throw new AppError('Unauthorized', 401);

  try {
    const payload = jwt.verify(token, process.env.JWT_PUBLIC_KEY!, { algorithms: ['RS25...
    req.user = payload as JwtPayload;
    next();
  } catch {
    throw new AppError('Token expired or invalid', 401, 'TOKEN_INVALID');
  }
};

// Role guard factory
export const requireRole = (...roles: UserRole[]) => {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!roles.includes(req.user.role)) {
      throw new AppError('Forbidden', 403, 'INSUFFICIENT_PERMISSIONS');
    }
    next();
  };
};

// Usage in router
router.get('/colleges', authenticate, requireRole('SUPER_ADMIN'), collegeController.list);
router.get('/colleges/:id', authenticate, requireRole('SUPER_ADMIN', 'COLLEGE_ADMIN'), ...
```

3.2 Pagination Standard

```
// All list endpoints use this standard
interface PaginationQuery {
  page?: number;    // default: 1
  limit?: number;   // default: 20, max: 100
  search?: string;
  sortBy?: string;
  sortOrder?: 'asc' | 'desc';
}

// Response meta
```

```
interface PaginationMeta {
  page: number;
  limit: number;
  total: number;
  totalPages: number;
  hasNext: boolean;
  hasPrev: boolean;
}
```

3.3 File Upload Pattern

```
// Presigned URL upload flow (preferred for large files)
// Step 1: Client requests presigned URL
POST /v1/uploads/presign
Body: { fileType: 'video/mp4', fileSize: 524288000, context: 'lesson-video' }
Response: { uploadUrl: 'https://s3.../...?X-Amz-Signature=...', fileKey: 'videos/...' }

// Step 2: Client uploads directly to S3 (no server involvement)
PUT {uploadUrl} with file binary

// Step 3: Client confirms upload to API
POST /v1/uploads/confirm
Body: { fileKey: 'videos/...', lessonId: '...' }
Response: { lesson: { ..., videoStatus: 'PROCESSING' } }
```

---

4. Certificate PDF Specification

The certificate PDF is generated server-side using PDFKit (Node.js).

Certificate Layout (A4 Landscape):

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■  [KodetoCareer Logo]              [Decorative border]■
■                                                      ■
■          CERTIFICATE OF COMPLETION                   ■
■          ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■                  ■
■               This is to certify that              ■
■                                                      ■
■             [STUDENT FULL NAME]                      ■
■        (in large, elegant typography)                ■
■                                                      ■
■    has successfully completed the course             ■
■                                                      ■
■     Python Programming with AI, Machine Learning   ■
■                 & Data Science                       ■
■                                                      ■
■   Conducted by KodetoCareer in collaboration with  ■
■             [College Name]                           ■
■                                                      ■
■   Duration: 4 Months    Date: DD MMM YYYY          ■
```

```
■                                                    ■
■  [QR Code]              [Authorised Signature]     ■
■  Certificate ID:        [Trainer Name]             ■
■  KTC-CERT-2024-XXXXX    [Designation]              ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

PDF Technical Specs:

- Page size: A4 Landscape (297mm × 210mm)
- Fonts embedded: Inter (regular, bold), Great Vibes (student name)
- Colours: #1E3A8A (primary blue), #F59E0B (gold accent)
- QR code: links to `https://verify.kodetocareer.com/{certificate_code}`
- File size target: < 500KB per certificate

---

5. Push Notification Payload Specs

All push notifications follow this Firebase FCM structure:

```
{
  "notification": {
    "title": "Quiz Result Available",
    "body": "You scored 85/100 in Python Basics Quiz!"
  },
  "data": {
    "type": "QUIZ_RESULT",
    "entityId": "quiz-attempt-uuid",
    "screen": "/quizzes/{quizId}/results",
    "timestamp": "2024-01-15T18:30:00Z"
  },
  "android": {
    "notification": {
      "channel_id": "academic_alerts",
      "priority": "HIGH",
      "color": "#1E3A8A"
    }
  },
  "apns": {
    "payload": {
      "aps": {
        "badge": 1,
        "sound": "default"
      }
    }
  }
}
```

Notification Channels (Android):

- `academic_alerts` — Quiz results, assignment grades (HIGH priority)

- `general` — Announcements, new content (DEFAULT priority)
- `system` — Attendance, certificates (DEFAULT priority)

---

6. CSV Import Specification

Student Bulk Import Format

```
Full Name,Email,Phone,Enrollment Number,Branch,Graduation Year
Rahul Gupta,rahul.gupta@example.com,9876543210,LNCT/CS/2024/001,Computer Science,2025
Priya Sharma,priya.sharma@example.com,9876543211,LNCT/CS/2024/002,Information Technolog...
```

Validation Rules per Column:

- Full Name: required, 3–100 chars, letters and spaces only
- Email: required, valid email format, unique in system
- Phone: required, 10 digits, valid Indian mobile number
- Enrollment Number: optional, max 50 chars
- Branch: optional, max 100 chars
- Graduation Year: optional, integer between 2020–2035

Import Response:

```
{
  "success": true,
  "data": {
    "totalRows": 50,
    "imported": 47,
    "failed": 3,
    "errors": [
      { "row": 15, "email": "bad-email", "reason": "Invalid email format" },
      { "row": 28, "email": "duplicate@test.com", "reason": "Email already registered" },
      { "row": 43, "email": "", "reason": "Email is required" }
    ]
  }
}
```

---

7. Analytics Calculation Specs

7.1 Placement Readiness Score (0–100)

```
Score = (
  attendance_pct * 0.20 +        // 20 points max
```

```
  quiz_avg_pct * 0.25 +          // 25 points max
  assignment_completion * 0.15 + // 15 points max
  profile_completeness * 0.10 +  // 10 points max
  activity_streak_score * 0.10 + // 10 points max
  skills_count_score * 0.10 +    // 10 points max
  course_completion_pct * 0.10   // 10 points max
)

Where:
  activity_streak_score = min(current_streak_days / 30, 1) * 100
  skills_count_score = min(skills_count / 10, 1) * 100
  profile_completeness = (filled_fields / total_profile_fields) * 100
```

7.2 Student Activity Streak

```
Calculated daily at midnight:
- streak = 0 on day of calculation
- For each preceding day (newest first):
  - If student has any activity_log entry on that date: streak++
  - If no activity: break

Maximum displayed streak: 365 days
Streak reset: If no activity today AND no activity yesterday
```

---

8. Environment Setup (Developer Quickstart)

```
# Prerequisites
node --version    # v20+
npm --version     # v10+
psql --version    # PostgreSQL 15+
redis-cli --version # Redis 7+

# Clone and install
git clone https://github.com/kodetocareer/api.git
cd kodetocareer-api
npm install

# Environment
cp .env.example .env
# Fill in .env with local values

# Database
npx prisma migrate dev
npx prisma db seed

# Run development server
npm run dev
# API runs on http://localhost:3000

# Run tests
```

```
npm test
npm run test:coverage

# Docker alternative (all services)
docker-compose up -d
# Starts: API + PostgreSQL + Redis
```

---

9. Git Workflow

Branch Strategy:

```
main          ← Production (protected, requires PR + review)
develop       ← Integration branch (auto-deploy to staging)
feature/*     ← Feature branches (from develop)
fix/*         ← Bug fix branches
hotfix/*      ← Emergency production fixes (from main)
```

Commit Message Format (Conventional Commits):

```
feat(quizzes): add auto-save every 30 seconds
fix(auth): refresh token not rotating on mobile
docs(api): update attendance endpoint documentation
chore(deps): update Prisma to 5.8.1
```

PR Requirements:

- All tests passing
- No TypeScript errors
- At least 1 reviewer approval
- Description includes: what changed, why, how to test

---

10. Testing Strategy

Backend Tests (Jest + Supertest)

```
// Integration test example
describe('POST /v1/auth/login', () => {
  it('returns 200 and tokens for valid credentials', async () => {
    const res = await request(app)
      .post('/v1/auth/login')
      .send({ email: 'test@test.com', password: 'password123' });

    expect(res.status).toBe(200);
    expect(res.body.data).toHaveProperty('accessToken');
```

```
    expect(res.body.data).toHaveProperty('user');
  });

  it('returns 401 for invalid password', async () => {
    const res = await request(app)
      .post('/v1/auth/login')
      .send({ email: 'test@test.com', password: 'wrongpassword' });

    expect(res.status).toBe(401);
    expect(res.body.success).toBe(false);
  });
});
```

Coverage Targets:

- Services: > 80% coverage
- Controllers: > 60% coverage
- Utils: > 90% coverage

Flutter Tests

- Unit tests for providers and repositories
- Widget tests for critical screens (login, quiz, attendance)
- Golden tests for design consistency on key screens

End of Development Specification